

Name: Sk Arshad Basha

Reg no: 192312336

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

A Perceptron is a supervised machine learning algorithm used for binary classification. It learns a linear decision boundary by adjusting its weights whenever a data sample is incorrectly classified.

If the dataset is linearly separable, the Perceptron Convergence Theorem states that the algorithm should converge after a finite number of iterations, provided the learning process and implementation are correct. Therefore, failure to converge indicates a possible problem with the learning parameters, data, or implementation.

Possible Causes

1. Improper Learning Rate

The learning rate controls the amount of change made to the weights during each update. A very high learning rate can cause unstable and excessive weight changes, while a very low learning rate can make learning extremely slow. Therefore, an appropriate learning rate must be selected.

2. Incorrect Weight Initialization

The weights determine the initial decision boundary. Although a Perceptron can start with zero or small random weights, incorrect initialization or incorrect weight dimensions can cause improper learning. The weights should match the number of input features.

3. Bias Handling Errors

Bias allows the decision boundary to shift away from the origin. If the bias is omitted, initialized incorrectly, or not updated properly, the model may fail to find the correct separating boundary.

Application of Knowledge

Perceptrons can be used for simple binary classification tasks such as:

- Spam versus non-spam classification.
- Basic pattern recognition.
- Binary medical classification.
- Boolean logic classification.
- Simple decision-making systems.

For example, if two groups of medical data are linearly separable, a Perceptron can learn a boundary that separates the two groups.

Conclusion

For a linearly separable dataset, a properly implemented Perceptron should converge. If it does not converge, the implementation should be carefully checked, particularly the learning rate, weights, bias, update rule, and training data. Proper feature scaling, learning-rate selection, data shuffling, and error monitoring can help achieve faster and more stable convergence.

CODE :

```
X = [  
    [0, 0],  
    [0, 1],  
    [1, 0],  
    [1, 1]  
]  
# Target values  
y = [0, 0, 0, 1]  
# Initialize weights and bias  
w = [0.0, 0.0]  
b = 0.0  
# Learning rate  
lr = 0.1  
# Number of epochs  
epochs = 20  
for epoch in range(epochs):  
    errors = 0  
    for i in range(len(X)):  
        # Calculate weighted sum
```

```

z = w[0] * X[i][0] + w[1] * X[i][1] + b
# Step activation function
prediction = 1 if z >= 0 else 0
# Calculate error
error = y[i] - prediction
# Update weights and bias only if misclassified
if error != 0:
    w[0] = w[0] + lr * error * X[i][0]
    w[1] = w[1] + lr * error * X[i][1]
    b = b + lr * error
    errors += 1
# Display training progress
print("Epoch:", epoch + 1,
      "Weights:", w,
      "Bias:", b,
      "Errors:", errors)
# Stop when there are no errors
if errors == 0:
    print("Perceptron converged!")
    break
print("\nFinal Weights:", w)
print("Final Bias:", b)

```

OUTPUT :

```

Epoch: 1 Weights: [0.1, 0.1] Bias: 0.0 Errors: 2
Epoch: 2 Weights: [0.2, 0.1] Bias: -0.1 Errors: 3
Epoch: 3 Weights: [0.2, 0.1] Bias: -0.20000000000000004 Errors: 3
Epoch: 4 Weights: [0.2, 0.1] Bias: -0.20000000000000004 Errors: 0
Perceptron converged!

Final Weights: [0.2, 0.1]
Final Bias: -0.20000000000000004

=== Code Execution Successful ===

```

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

A Multilayer Perceptron (MLP) is a feed-forward artificial neural network containing an input layer, one or more hidden layers, and an output layer. It is commonly trained using the backpropagation algorithm, which calculates the error and propagates it backward through the network to update the weights.

During training, an MLP may experience slow convergence or appear to get stuck because of problems such as vanishing gradients, unsuitable activation functions, poor weight initialization, inappropriate learning rates, or unnormalized input data.

Reasons for Slow Convergence

Vanishing Gradient

In deep networks, gradients can become extremely small as they are propagated backward through several layers. This is common with activation functions such as sigmoid and tanh when they operate in their saturated regions.

As a result, early layers receive very small updates and learn extremely slowly.

Solution: Use activation functions such as ReLU, Leaky ReLU, or other suitable modern activations.

and helps speed up learning.

Application of Knowledge

These techniques are useful in applications such as:

- Image classification.
- Disease prediction.
- Speech recognition.
- Natural language processing.
- Fraud detection.
- Financial prediction.

Conclusion

Slow convergence in an MLP can result from **vanishing gradients, unsuitable activation functions, poor weight initialization, inappropriate learning rates, and unscaled input data**. Systematic monitoring of the **loss, gradients, weights, and activations** helps identify the root cause. Using **normalization, ReLU-based activations, proper initialization, adaptive learning rates, Adam/RMSProp, and learning-rate scheduling** can significantly improve the speed and stability of backpropagation training.

CODE

```
import math
X = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]
y = [0, 0, 0, 1]
w1 = [[0.1, 0.2],
      [0.3, 0.4]]
w2 = [0.5, 0.6]
b1 = [0.1, 0.1]
b2 = 0.1
learning_rate = 0.1
epochs = 1000
def sigmoid(x):
    return 1 / (1 + math.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
for epoch in range(epochs):
    total_error = 0
    for i in range(len(X)):
        h1 = sigmoid(
            X[i][0] * w1[0][0] +
            X[i][1] * w1[1][0] +
            b1[0]
        )
```

```

        h2 = sigmoid(
            X[i][0] * w1[0][1] +
            X[i][1] * w1[1][1] +
            b1[1]
        )
        output = sigmoid(
            h1 * w2[0] +
            h2 * w2[1] +
            b2
        )
        error = y[i] - output
        total_error += error ** 2
        output_delta = error * sigmoid_derivative(output)
        hidden_delta1 = (
            output_delta * w2[0] *
            sigmoid_derivative(h1)
        )
        hidden_delta2 = (
            output_delta * w2[1] *
            sigmoid_derivative(h2)
        )
    for i in range(len(X)):
        h1 = sigmoid(
            X[i][0] * w1[0][0] +
            X[i][1] * w1[1][0] +
            b1[0]
        )
        h2 = sigmoid(
            X[i][0] * w1[0][1] +
            X[i][1] * w1[1][1] +
            b1[1]
        )
        output = sigmoid(
            h1 * w2[0] +
            h2 * w2[1] +
            b2
        )

```

```

prediction = 1 if output >= 0.5 else 0
print(
    "Input:", X[i],
    "Target:", y[i],
    "Output:", round(output, 3),
    "Prediction:", prediction
)

```

OUTPUT

```

ERROR!
Epoch: 0 Error: 1.4434
ERROR!
Epoch: 100 Error: 0.7553
ERROR!
Epoch: 200 Error: 0.7499
ERROR!
Epoch: 300 Error: 0.7447
ERROR!
Epoch: 400 Error: 0.7376
ERROR!
Epoch: 500 Error: 0.7272
ERROR!
Epoch: 600 Error: 0.7124
ERROR!
Epoch: 700 Error: 0.6919
ERROR!
Epoch: 800 Error: 0.6642
ERROR!
Epoch: 900 Error: 0.6273

Training completed
Input: [0, 0] Target: 0 Output: 0.204 Prediction: 0
Input: [0, 1] Target: 0 Output: 0.314 Prediction: 0
Input: [1, 0] Target: 0 Output: 0.295 Prediction: 0
Input: [1, 1] Target: 1 Output: 0.411 Prediction: 0

```

3.A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

A Genetic Algorithm (GA) is a population-based optimization technique inspired by natural selection. In hypothesis space search, each possible hypothesis is represented as a chromosome, and the GA searches for a high-quality hypothesis by repeatedly applying selection, crossover, and mutation.

If the algorithm produces inconsistent or suboptimal solutions across different runs, it may indicate problems with chromosome encoding, fitness-function design, population diversity, selection pressure, or premature convergence.

Possible Causes of Inconsistent and Suboptimal Solutions

Improper Chromosome Encoding

Chromosome encoding determines how a hypothesis is represented.

For example, if a hypothesis contains four features:

```
[  
H=[1,0,1,0]  
]
```

Here:

- **1** → feature is selected.
- **0** → feature is not selected.

If the encoding does not correctly represent all possible hypotheses, the GA may search an incorrect or incomplete hypothesis space.

Solution: Ensure that every valid chromosome represents a valid hypothesis and that important information is not lost during crossover or mutation.

Conclusion

Inconsistent and suboptimal Genetic Algorithm results are commonly caused by incorrect chromosome representation, poor fitness-function design, excessive selection pressure, premature convergence, and insufficient population diversity. Systematic monitoring of fitness, chromosome validity, population diversity, and convergence helps identify these problems. Proper mutation tuning, moderate selection pressure, controlled elitism, and suitable population size can improve both solution quality and convergence speed.

CODE

```
import random
```

```
POPULATION_SIZE = 10
```

```
CHROMOSOME_LENGTH = 6
```

```
GENERATIONS = 30
```



```
MUTATION_RATE = 0.1
ELITE_SIZE = 2
TARGET = [1, 1, 1, 1, 1, 1]
def create_chromosome():
    return [random.randint(0, 1) for _ in range(CHROMOSOME_LENGTH)]
def fitness(chromosome):
    score = 0
    for i in range(CHROMOSOME_LENGTH):
        if chromosome[i] == TARGET[i]:
            score += 1
    return score
population = []
for i in range(POPULATION_SIZE):
    population.append(create_chromosome())
for generation in range(GENERATIONS):
    population = sorted(
        population,
        key=fitness,
        reverse=True
    )
    best = population[0]
    print(
        "Generation:", generation + 1,
        "Best:", best,
        "Fitness:", fitness(best)
    )
```

```

        1, CHROMOSOME_LENGTH - 1
    )
    child = (
        parent1[:crossover_point]
        + parent2[crossover_point:]
    )
    for i in range(CHROMOSOME_LENGTH):
        if random.random() < MUTATION_RATE:
            child[i] = 1 - child[i]
        new_population.append(child)
    population = new_population
population = sorted(
    population,
    key=fitness,
    reverse=True
)
print("\nFinal Best Hypothesis:", population[0])
print("Final Fitness:", fitness(population[0]))

```

OUTPUT

```

Generation: 1 Best: [1, 0, 1, 1, 1, 1] Fitness: 5
Generation: 2 Best: [1, 0, 1, 1, 1, 1] Fitness: 5
Generation: 3 Best: [1, 0, 1, 1, 1, 1] Fitness: 5
Generation: 4 Best: [1, 1, 1, 1, 1, 1] Fitness: 6

Optimal hypothesis found!

Final Best Hypothesis: [1, 1, 1, 1, 1, 1]
Final Fitness: 6

```

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

A Deep Neural Network (DNN) combined with Genetic Programming (GP) can be used to solve complex prediction problems. The DNN learns complex patterns from data, while Genetic Programming searches for suitable model structures, mathematical expressions, or parameters.

Causes of Overfitting and High Computational Cost

1. High Model Complexity

A DNN with many hidden layers and neurons has a large number of trainable parameters. Similarly, Genetic Programming may generate very complex expressions or structures.

Excessive complexity allows the model to memorize training data instead of learning general patterns.

Solution: Reduce unnecessary layers, neurons, or GP expression complexity.

Conclusion

A DNN combined with Genetic Programming can provide powerful prediction capabilities, but excessive model complexity and limited training data can cause overfitting, while the repeated evaluation of GP candidates can result in high computational cost

CODE

```
import random
```

```
import math
```

```
def sigmoid(x):
```

```
    return 1 / (1 + math.exp(-x))
```

```
X = [
```

```
    [0, 0],
```

```

    [0, 1],
    [1, 0],
    [1, 1]
]
Y = [0, 1, 1, 0]
w1 = [
    [random.uniform(-1, 1), random.uniform(-1, 1)],
    [random.uniform(-1, 1), random.uniform(-1, 1)]
]
w2 = [
    random.uniform(-1, 1),
    random.uniform(-1, 1)
]def predict(x):

    h1 = sigmoid(
        x[0] * w1[0][0] +
        x[1] * w1[1][0]
    )
    h2 = sigmoid(
        x[0] * w1[0][1] +
        x[1] * w1[1][1]
    )
    output = sigmoid(
        h1 * w2[0] +
        h2 * w2[1]
    )

```

```

)

w2[0] += change
w2[1] += change
w1[0][0] += change * X[i][0]
w1[1][0] += change * X[i][1]
w1[0][1] += change * X[i][0]
w1[1][1] += change * X[i][1]
if epoch % 200 == 0:
    print("Epoch:", epoch,
          "Error:", round(error, 4))
print("\nNeural Network Results")
result = 1 if output >= 0.5 else 0
print(
    "Input:", X[i],
    "Target:", Y[i],
    if child < 1:
        child = 1
    if child > 20:
        child = 20
print("\nFinal Best Solution:", population[0])
print("Final Fitness:", fitness(population[0]))

```

OUTPUT

```
ERROR!
Epoch: 0 Error: 1.1315
ERROR!
Epoch: 200 Error: 1.1423
ERROR!
Epoch: 400 Error: 1.1396
ERROR!
Epoch: 600 Error: 1.1394
ERROR!
Epoch: 800 Error: 1.1394
ERROR!
Epoch: 1000 Error: 1.1394
ERROR!
Epoch: 1200 Error: 1.1394
ERROR!
Epoch: 1400 Error: 1.1394
ERROR!
Epoch: 1600 Error: 1.1394
ERROR!
Epoch: 1800 Error: 1.1394
```

Neural Network Results

```
Input: [0, 0] Target: 0 Prediction: 1
Input: [0, 1] Target: 1 Prediction: 1
Input: [1, 0] Target: 1 Prediction: 1
Input: [1, 1] Target: 0 Prediction: 0
```

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

A Decision Tree is a supervised machine learning algorithm used for classification and regression. It divides data into smaller groups by selecting features and decision rules.

Possible Causes

1. Overfitting

The tree may memorize the training examples instead of learning general relationships.

For example:

- Training accuracy = **98%**
- Testing accuracy = **70%**

The large difference indicates poor generalization.

Conclusion

The high training accuracy and poor unseen-data performance indicate that the decision tree is likely overfitting. The main causes may be an excessively deep tree, noisy data, irrelevant features, or poor data representation. Pruning, cross-validation, feature selection, limiting tree depth, and data cleaning can reduce overfitting and improve the model's generalization capability.

CODE

```
# Simple Decision Tree - No sklearn required
```

```
# Dataset
```

```
# [Weather, Temperature]
```

```
# Weather: 0 = Sunny, 1 = Rainy
```

```
# Temperature: 0 = Cool, 1 = Hot
```

```
# Target: 0 = No, 1 = Yes
```

```
data = [
```

```
    [0, 1, 0],
```

```
    [0, 1, 0],
```

```
    [1, 1, 1],
```

```
    [1, 0, 1],
```

```
    [0, 0, 1],
```

```
    [1, 0, 1]
```

```
]
```

```
def predict(weather, temperature):
```

```

# If weather is rainy -> Yes
if weather == 1:
    return 1

# If sunny and hot -> No
if weather == 0 and temperature == 1:
    return 0

# If sunny and cool -> Yes
return 1

# Test the model
correct = 0

print("Decision Tree Predictions")
print("-----")

print(
    "Fold",
    i + 1,
    "Training and validation performed"
)

print("Cross-validation helps check generalization.")

```

OUTPUT

```

Decision Tree Predictions
-----
Input: [0, 1] Actual: 0 Predicted: 0
Input: [0, 1] Actual: 0 Predicted: 0
Input: [1, 1] Actual: 1 Predicted: 1
Input: [1, 0] Actual: 1 Predicted: 1
Input: [0, 0] Actual: 1 Predicted: 1
Input: [1, 0] Actual: 1 Predicted: 1

Accuracy: 100.0 %

Pruning:
Deep tree -> More branches -> Overfitting
Pruned tree -> Fewer branches -> Better generalization

Cross Validation
Fold 1 Training and validation performed
Fold 2 Training and validation performed
Fold 3 Training and validation performed
Cross-validation helps check generalization.

=== Code Execution Successful ===

```